CS/ECE 752
Fall 2025
Homework 4
Due 11 AM Central Time on Friday, October 3rd, 2025

You should do this assignment on your own, although you are encouraged to talk with classmates electronically or on Piazza about any issues you may have encountered. The standard late assignment policy applies: you may submit up to 1 day (24 hours) late without penalty. Moreover, please consult the homework page for details about LLM usage on homework assignments. In particular, recall these are practice problems for Midterm 1 – using LLMs to solve all of them without understanding the underlying concepts will likely leave you ill-prepared for Midterm 1.

What to Hand In

To submit your assignment, please type up your answers to the following questions and submit **one PDF named HW4-<netID>.pdf** on Canvas. If you prefer writing your answers by hand, that is fine, but please scan your solutions and submit them on Canvas. **Please make sure to put your name in the top-right of the first page of the PDF. And please have the answer to each question start on a new page to simplify grading.** Note that just because each question starts on a new page, it does not mean you need to write a whole page of answer – the page breaks are there for simplifying grading.

Total Points: 68

Problem 1 [26 points]

Consider the following architecture.

Functional Unit Type	Cycles in EX	Number of Functional Units	Pipelined
Integer	1	1	No
FP Add/Subtract	3	1	Yes
FP/Integer Multiplier	6	1	Yes
FP/Integer Divider	24	1	No

- In this problem we will use MIPS pipeline.
- The integer functional unit performs integer addition (including effective address calculation for loads/stores), subtraction, logic operations and branch operations.
- There is full forwarding and bypassing, including forwarding from the end of an FU to the MEM stage for stores.
- Loads and stores complete in one cycle. That is, they spend one cycle in the MEM stage after the effective address calculation.
- There are as many registers, both FP and integer, as you need.
- There is one branch delay slot.
- While the hardware has full forwarding and bypassing, it is the responsibility of the compiler to schedule such that the operands of each instruction are available when needed by each instruction.
- If multiple instructions finish their EX stages in the same cycle, then we will assume they can all proceed to the MEM stage together. Similarly, if multiple instructions finish their MEM stages in the same cycle, then we will assume they can all proceed to the WB stage together. In other words, for the purpose of this problem, you are to ignore structural hazards on the MEM and WB stages.

The following code implements the DAXPY operation, $Y = aX + Y$, for a vector length 100. Initially, R1 is set to the base address of array X and R2 is set to the base address of Y. Assume initial value of R3 = 0.

```
        DADDIU R4, R1, #800
FOO:    L.D F2, 0(R1)
        MUL.D F4, F2, F0
        L.D F6, 0(R2)
        ADD.D F6, F4, F6
        S.D F6, 0(R2)
        DADDIU R1, R1, #8
        DADDIU R2, R2, #8
        DSLTU R3, R1, R4 // set R3 to 1 if R1 < R4
        BNEZ R3, FOO
```

Part A [6 points]

Consider the role of the compiler in scheduling the code. Rewrite this loop, but let every row take a cycle. If an instruction can't be issued on a given cycle (because the current instruction has a dependency that will not be resolved in time), write STALL instead, and move on to the next cycle to see if it can be issued then. Assume that a NOP is scheduled in the branch delay slot (effectively stalling 1 cycle after the branch). Explain all stalls, but don't reorder instructions. How many cycles elapse before the second iteration begins? Show your work.

Part B [6 points]

Now reschedule the loop. You can change immediate values and memory offsets. You can reorder instructions, but don't change anything else. Show any stalls that remain. How many cycles elapse before the second iteration begins? Show your work.

Part C [6 points]

Now unroll and reschedule the loop the minimum number of times needed to eliminate all stalls. You can remove redundant instructions. How many times did you unroll the loop? How many cycles elapse before the next iteration of the loop begins? Don't worry about clean-up code. Show your work.

Part D [8 points]

Consider a VLIW processor in which one instruction can support two memory operations (load or store), one integer operation (addition, subtraction, comparison, or branch), one floating point add or subtract, and one floating point multiply or divide. There is no branch delay slot. Now unroll the loop four times, and schedule it for this VLIW to take as few stall cycles as possible. How many cycles do the four iterations take to complete? Use the following table template to show your work.

(You may use a different number of lines in the table template than are provided)

MEM 1	MEM 2	INTEGER	FP ADD	FP MUL

Problem 2 [14 points]

Part A [6 points]

Assume you are an engineer at a company, and you are responsible for designing a new accelerator your company would like to integrate into their future hardware designs. Since this is a brand new accelerator, which has no prior versions and no existing tools to model its behavior, your boss asks you suggest what tool is appropriate to analyze the behavior of your accelerator/future hardware for a set of programs P_i (where i represents program i). Do you suggest an analytical model, hardware prototype (i.e., RTL simulation, taping out a chip, or an FPGA prototype), trace-driven simulator, or an execution-driven simulator? To receive credit you must justify your answer.

Part B [5 points]

Describe one way that increasing clock frequency may improve an application's performance, and one way that increasing clock frequency may hurt an application's performance. To receive credit, you must justify your answers.

Part C [3 points]

Suppose we have a deeply pipelined processor, for which we implement a branch-target buffer (BTB) for the conditional branches only. Assume that the misprediction penalty is always five cycles, and the buffer miss penalty is always four cycles. Assume a 91% hit rate, 90% accuracy, and 25% branch frequency. How much faster is the processor with the BTB versus a processor that has a fixed two-cycle branch penalty? Assume a base clock cycle per instruction (CPI) without branch stalls of one.

Problem 3 [10 points]

Consider the following C code fragment:

```
for (i = 0; i < 100; i++) {
        if (c == 0) {
                …
                c = …;
                … // code I
        }
        else {
                …
                c = …;
                … // code II
        }
        … // code III
}
```

The above translates to the MIPS fragment below. R5 and R6 store variables i and c, respectively.

```
Init:   MOV.D  R5, R0      // i = 0
If:     BNEZ   R6, Else    // Branch1 (c == 0?)
        ...                // Code I = 10 instructions; contains a write to R6
        J Join
Else:
        ...                // Code II = 100 instructions; contains a write to R6

Join:   ...                // Code III = 10 instructions

Loop:   DADDI  R5, R5, #1  // i++
        DSUBI  R7, R5, #100
        BNEZ   R7, If      // Branch2 (i == 100?)
        J Done
```

(continued on next page)

Problem 3 (Continued)

Suppose the segments “Code I” (if part), “Code II” (else part), and Code III (common part) contain 10, 100, and 10 assembly instructions respectively. You did a profile run of this program and found that on average, Branch1 is taken once in 100 iterations of the “for loop”.

Your boss suggests that you perform one of the following two transformations to speed up the above code: (1) Loop unrolling with an unrolling factor of 2. (2) Trace scheduling.

Which one of these would be more effective and why? Show the code with the more effective transformation applied. If you use trace scheduling, then include any repair code and branches into and out of it. Assume that only the values of c and i may need repair. Assume that registers R10 and higher are free for your use.

Problem 4 [18 points]

Consider the following MIPS code. The register R0 is always 0.

```
        DADD    R1, R0, R0

L1:     DADD    R2, R0, R0

L2:     DADDI   R2, R2, #1
        DSUBI   R3, R2, #3
        BNEQZ   R3, L2          -- Branch 1

        DADDI   R1, R1, #1
        DSUBI   R4, R1, #4
        BNEQZ   R4, L1          -- Branch 2
```

Each table below refers to only one branch. For instance, branch 1 will be executed 12 times. Those 12 times should be recorded in the table for branch 1. Similarly, branch 2 is executed 4 times.

Part A [6 points]
Assume that 1-bit branch predictors are used. When the processor starts to execute the above code, both predictors contain value N (Not taken). What is the number of correct predictions? Use the following tables to record the prediction and action of each branch. Several entries are filled in for you.

Branch 1:

Step	Branch 1 Prediction	Actual Branch 1 Action
1	N	T
2		T
3		
4		
5		
6		
7		
8		
9		
10		
11		
12		

Branch 2:

Step	Branch 2 Prediction	Actual Branch 2 Action
1	N	T
2		
3		
4		

Part B [6 Points]
Now assume that 2-bit saturation counters are used. When the processor starts to execute the above code, both counters contain value 00. What is the number of correct predictions? Use the following tables to record the prediction and action of each branch. You must follow the 2-bit saturation counters taught in class for branch prediction.

Branch 1:

Step	Counter Value	Branch 1 Prediction	Actual Branch 1 Action
1	0 0	N	T
2			T
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

Branch 2:

Step	Counter Value	Branch 2 Prediction	Actual Branch 2 Action
1	0 0	N	T
2			
3			
4			

Part C [6 points]

Now assume that 2 level *global* correlating predictors of the form (2, 1) are used. (Note: similar to class, here global means that the history used captures the history of all previous branches.) When the processor starts to execute the above code, the outcome of the previous two branches is not taken (N). Also assume that the initial state of predictors of all branches is not taken (N). What is the number of correct predictions? Use the following table to record your steps. Record the "New State" of predictors in the form W/X/Y/Z where:

W - state corresponds to the case where the last branch and the branch before the last are both TAKEN.

X - state corresponds to the case where the last branch is TAKEN and the branch before the last is NOT TAKEN.

Y - state corresponds to the case where the last branch is NOT TAKEN and the branch before the last is TAKEN.

Z - state corresponds to the case where the last branch and the branch before the last are both NOT TAKEN.

This means that the table for either branch looks like this for the initial state (**study this table well**):

Index		Letter
00	N	Z
01	N	X
10	N	Y
11	N	W

Note: The state of the predictor at position W/X/Y/Z can be *N* for predict-not-taken or *T* for predict-taken.

(tables are on next page)

Branch 1:

Step	Branch 1 Prediction	Actual Branch 1 Action	New State
1	N	T	N/N/N/T
2		T	
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			

Branch 2:

Step	Branch 2 Prediction	Actual Branch 2 Action	New State
1	N	T	N/N/T/N
2			
3			
4			